Lab: Static Type Checking

Learning Goals

The primary goals of this lab are:

- Programming with higher-order functions.
- Static type checking and understanding the interplay between type checking and evaluation.

Functional Programming Skills Higher-order functions with collections and callbacks.
Programming Language Ideas Static type checking and type safety. Records.

Instructions

A version of project files for this lab resides in the public pppl-lab4 repository. Please follow separate instructions to get a private clone of this repository for your work.

You will be replacing `???` or `case _ => ???` in the `Lab4.scala` file with solutions to the coding exercises described below.

Your lab will not be graded if it does not compile. You may check compilation with your IDE, `sbt compile`, or with the "sbt compile" GitHub Action provided for you. Comment out any code that does not compile or causes a failing assert. Put in `???` as needed to get something that compiles without error.

You may add additional tests to the `Lab4Spec.scala` file. In the `Lab4Spec.scala`, there is empty test class `Lab4StudentSpec` that you can use to separate your tests from the given tests in the `Lab4Spec` class. You are also likely to edit `Lab4.worksheet.sc` for any scratch work. You can also use `Lab4.worksheet.ts` to write and experiment in a JavaScript file that you can then parse into a TypeScripty AST (see `Lab4.worksheet.sc`).

Static Typing: TypeScripty: Functions and Objects

Static Typing

As we have seen in the prior labs, dealing with coercions and checking for dynamic type errors complicate the interpreter implementation (i.e., `step`). Some languages restrict the possible programs that it will execute to ones that it can guarantee will not result in a dynamic type error. This restriction of programs is enforced with an analysis phase after parsing but before evalation known as *type checking*. Such languages are called *statically-typed*. In this lab, we implement a statically-typed version of JavaScripty that we affectionately call TypeScripty. We will not permit *any* type coercions and simultaneously guarantee the absence of dynamic type errors.

Multi-Parameter Recursive Functions

Using our skills working with higher-order functions on collections from previous assignments, we now consider functions with zero-or-more parameters (instead of exactly one):

$$
\begin{array}{rrcl}
\text{types} & \tau & ::= & (\overline{y : \tau}) \Rightarrow \tau' \\
\text{values} & v & ::= & x^? (\overline{y : \tau}) \tau^? \Rightarrow e_1 \\
\text{expressions} & e & ::= & x^? (\overline{y : \tau}) \tau^? \Rightarrow e_1 \mid e_1 (e_2) \\
\text{optional variables} & x^? & ::= & x \mid \varepsilon \\
\text{optional type annotations} & \tau^? & ::= & : \tau \mid \varepsilon
\end{array}
$$

We write a sequence of things using either an overbar or dots (e.g., $\overline{y}$ or $y_1, \ldots, y_n$ for a sequence of variables). Functions can now take any number of parameters $\overline{y : \tau}$. We have a language of types τ and function parameters $\overline{y}$ are annotated with types $\overline{\tau}$.

Functions can be named or unnamed $x^?$ and can be annotated with a return type or unannotated $\tau^?$. To define recursive functions, the function needs to be named *and* annotated with a return type.

To represent an arbitrary number of function parameters or function call arguments in Scala, we use an appropriate `List`:

```
trait Typ  // t
case class TFun(yts: List[(String,Typ)], tret: Typ) extends Typ
  // t ::= (yts) => tret

trait Expr // e
case class Fun(xopt: Option[String], yts: List[(String,Typ)],
  tretopt: Option[Typ], e1: Expr) extends Expr
  // e ::= xopt(yts)tretopt => e1
```

```
case class Call(e0: Expr, es: List[Expr]) extends Expr
  // e ::= e0(es)
```

Immutable Objects (Records)

Similarly, we now consider immutable objects that can have an arbitrary number of fields:

$$\begin{array}{rrcl} \text{types} & \tau & ::= & \{\overline{f:\tau}\} \\ \text{values} & v & ::= & \{\overline{f:v}\} \\ \text{expressions} & e & ::= & \{\overline{f:e}\} \mid e_1.f \end{array}$$

An object literal expression

$$\{f_1:e_1,\ldots,f_n:e_n\}$$

is a comma-separated sequence of field names with initialization expressions surrounded by braces. Objects here are more like records in other programming languages compared to actual JavaScript objects, as we do not have any form of mutation or dynamic extension. Fields here correspond to what JavaScript calls properties but which can be dynamically added or removed from objects. We use the term fields to emphasize that they are fixed based on their type:

$$\{f_1:e_1,\ldots,f_n:e_n\} \quad : \quad \{f_1:\tau_1,\ldots,f_n:\tau_n\}$$

Note that an object value is an object literal expression where each field is a value:

$$\{f_1:v_1,\ldots,f_n:v_n\}$$

The field read expression $e_1.f$ evaluates e_1 to an object value and then looks up the field named f.

To represent object types and object literal expressions in Scala, we use an appropriate `Map`:

```
case class TObj(fts: Map[String, Typ]) extends Typ    // t ::= { fts }
case class Obj(fes: Map[String,Expr]) extends Expr    // e ::= { fes }
case class GetField(e1: Expr, f: String) extends Expr // e ::= e1.f
```

Otherwise, we consider our base JavaScripty language that has numbers with arithmetic expressions, booleans with logic and comparison expressions, strings with concatenation, **undefined** with printing, and **const**-variable declarations. In summary, the type language τ includes base types for numbers, booleans, strings, and **undefined**, as well as constructed types for functions and objects described above:

$$\begin{array}{rrcl} \text{types} & \tau & ::= & \texttt{number} \mid \texttt{bool} \mid \texttt{string} \mid \texttt{Undefined} \\ & & \mid & \texttt{(}\overline{y\texttt{:}\,\tau}\texttt{)}\ \texttt{=>}\ \tau' \mid \texttt{\{}\overline{f\texttt{:}\,\tau}\texttt{\}} \end{array}$$

As an aside, we have chosen a concrete syntax that is compatible with the TypeScript language that adds typing to JavaScript. TypeScript is a proper superset of JavaScript, so it is not as strictly typed as TypeScripty is here in this lab.

Interpreter Implementation

We break our interpreter implementation into evaluation and type checking.

Small-Step Reduction

For evaluation, we continue with implementing a small-step operational semantics with a `step` that implements a single reduction step $e \longrightarrow e'$ on closed expressions. Because of the static type checking, the reduction-step cases can be greatly simplified: we eliminate performing all coercions, and what's cool is that we no longer need to represent the possibility of a dynamic typeerror (e.g., with a `Either[DynamicTypeError,Expr]`).

We can use the more basic type signature for `step`:

```
def step(e: Expr): Expr = ???
```

corresponding to the more basic judgment form $e \longrightarrow e'$ (given in the subsequent sections).

To make easier to identify implementation bugs, we introduce another Scala exception type to throw when there is no possible next step.

```
case class StuckError(e: Expr) extends Exception
```

However, the intent of this exception is that it should get thrown at run-time! If it does get thrown, that signals a bug in our interpreter implementation rather than an error in the TypeScripty test input.

In particular, if the TypeScripty expression `e` passed into `step` is closed and well-typed (i.e., `inferType(e)` does not throw `StaticTypeError`), then `step` should never throw a `StuckError`. This property is *type safety*.

Recall that to implement `step`, we need to implement a substitution function `substitute` corresponding to $[v/x]e$ that we use to eagerly apply variable bindings:

```
def substitute(v: Expr, x: String, e: Expr) = ???
```

Static Type Checking

We implement a static type checker that up front rules out programs that would get stuck in taking reduction steps. This type checker is very similar to a big-step interpreter. Instead of computing the value of an expression by recursively computing the value of each sub-expression, we infer the type of an expression, by recursively inferring the type of each sub-expression. An expression is *well-typed* if we can infer a type for it.

Given its similarity to big-step evaluation, we formalize a type inference algorithm in a similar way. That is, we define the judgment form $\Gamma \vdash e : \tau$, which says, "In type environment Γ, expression e has type τ." We then implement a function `hastype`:

```
type TEnv = Map[String, Typ]
def hastype(tenv: TEnv, e: Expr): Typ = ???
```

that corresponds directly to this judgment form. It takes as input a type environment `tenv: TEnv` (Γ) and an expression `e: Expr` (e) to return a type `Typ` (τ). It is informative to compare the rules defining typing with a big-step operational semantics.

To signal a type error, we will use a Scala exception

```
case class StaticTypeError(tbad: Typ, esub: Expr, e: Expr) extends Exception
```

where `tbad` is the type that is inferred sub-expression `esub` of input expression `e`. These arguments are used to construct a useful error message. We also provide a helper function `err` to simplify throwing this exception.

While it is possible to implement iterative reduction via `step` and type inference via`hastype` independently, it is generally easier to "incrementally grow the language" by going language-feature by-language-feature for all functions rather than function-by-function. In the subsequent steps, we describe the small-step operational semantics and the static typing semantics together incrementally by language feature.

Notes

For testing your implementation, there are some interface functions defined that calls your `step` and `hastype` implementations with some debugging information:

- The `iterateStep: Expr => Expr` function repeatedly calls your `step` implementation until reaching a value.

- The `inferType: Expr => Typ` function calls your **`hastype`** function with an empty type environment.

Note that the provided tests are minimal. **You will want to add your own tests to cover most language features.**

Base TypeScripty

Small-Step Reduction

We consider the base TypeScripty that has numbers with arithmetic expressions, booleans with logic and comparison expressions, strings with concatenation, **undefined** with printing, and **const**-variable declarations from previous assignments and remove all coercions.

Exercise 0.1 (Small-Step Reduction for Base TypeScripty)**.** Implement `step` for base TypeScripty following the small-step operational semantics in Figure 1 defining the reduction-step judgment form $e \longrightarrow e'$.

Note that your task here is simpler than what you have done before in previous assignments. There are no judgment forms or rules defining coercions (e.g., `toBoolean`) or stepping to a typeerror result (e.g., `Left(DynamicTypeError(e))`).

You will need to implement a helper function `substitute` for base TypeScripty to perform scope-respecting substitution $[v/x]e$ as in previous assignments.

Notes

- You may use (or ignore) the provided helper function `doInequality` to implement the DoInequalityNumber and DoInequalityString rules.

Static Type Checking

We define static typing with the judgment form $\Gamma \vdash e : \tau$ of base TypeScripty that has numbers with arithmetic expressions, booleans with logic and comparison expressions, strings with concatenation, **undefined** with printing, and **const**-variable declarations.

Observe how closely the Type rules align with the Do rules in Figure 1, except for having a big-step evaluation structure with types.

Exercise 0.2 (Static Type Checking for Base TypeScripty)**.** Implement a function `hastype` for base TypeScript following the static typing semantics in Figure 2 defining the typing judgment form $\Gamma \vdash e : \tau$.

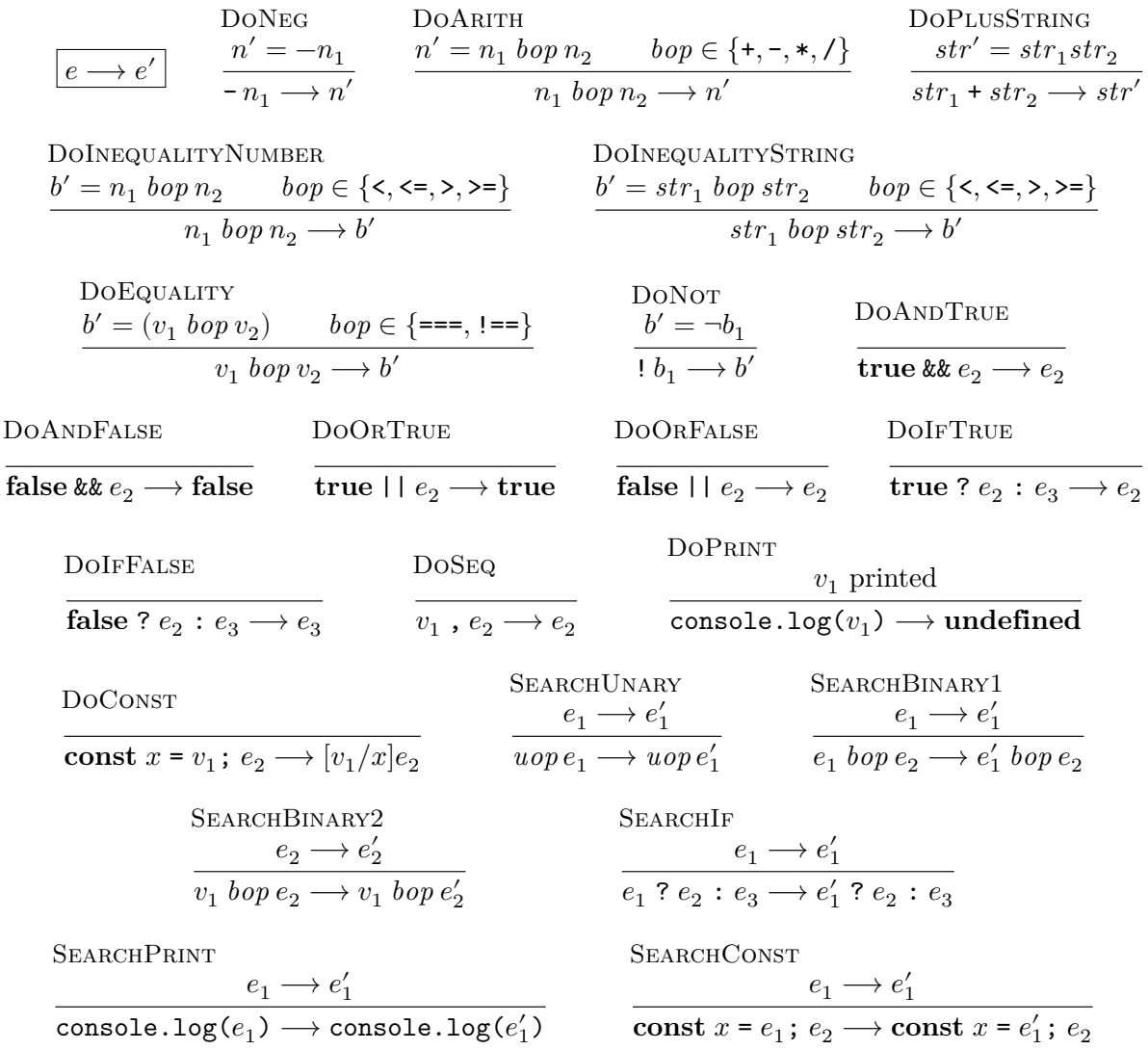


Figure 1: Small-step operational semantics of base TypeScripty, including numbers with arithmetic expressions, booleans with logic and comparison expressions, strings with concatenation, **undefined** with printing, and **const**-variable declarations.

$\boxed{\Gamma \vdash e : \tau}$

$$\textsc{TypeNumber}\ \frac{}{\Gamma \vdash n : \texttt{number}} \qquad \textsc{TypeString}\ \frac{}{\Gamma \vdash str : \texttt{string}} \qquad \textsc{TypeNeg}\ \frac{\Gamma \vdash e_1 : \texttt{number}}{\Gamma \vdash \texttt{-}\, e_1 : \texttt{number}}$$

$$\textsc{TypeArith}\ \frac{\Gamma \vdash e_1 : \texttt{number} \qquad \Gamma \vdash e_2 : \texttt{number} \qquad bop \in \{\texttt{+}, \texttt{-}, \texttt{*}, \texttt{/}\}}{\Gamma \vdash e_1\ bop\ e_2 : \texttt{number}}$$

$$\textsc{TypePlusString}\ \frac{\Gamma \vdash e_1 : \texttt{string} \qquad \Gamma \vdash e_2 : \texttt{string}}{\Gamma \vdash e_1 \texttt{ + } e_2 : \texttt{string}}$$

$$\textsc{TypeInequalityNumber}\ \frac{\Gamma \vdash e_1 : \texttt{number} \qquad \Gamma \vdash e_2 : \texttt{number} \qquad bop \in \{\texttt{<}, \texttt{<=}, \texttt{>}, \texttt{>=}\}}{\Gamma \vdash e_1\ bop\ e_2 : \texttt{bool}}$$

$$\textsc{TypeInequalityString}\ \frac{\Gamma \vdash e_1 : \texttt{string} \qquad \Gamma \vdash e_2 : \texttt{string} \qquad bop \in \{\texttt{<}, \texttt{<=}, \texttt{>}, \texttt{>=}\}}{\Gamma \vdash e_1\ bop\ e_2 : \texttt{bool}}$$

$$\textsc{TypeEquality}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau \qquad bop \in \{\texttt{===}, \texttt{!==}\}}{\Gamma \vdash e_1\ bop\ e_2 : \texttt{bool}} \qquad \textsc{TypeBool}\ \frac{}{\Gamma \vdash b : \texttt{bool}} \qquad \textsc{TypeNot}\ \frac{\Gamma \vdash e_1 : \texttt{bool}}{\Gamma \vdash \texttt{!}\, e_1 : \texttt{bool}}$$

$$\textsc{TypeAndOr}\ \frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : \texttt{bool} \qquad bop \in \{\texttt{\&\&}, \texttt{||}\}}{\Gamma \vdash e_1\ bop\ e_2 : \texttt{bool}}$$

$$\textsc{TypeIf}\ \frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 : \tau} \qquad \textsc{TypeUndefined}\ \frac{}{\Gamma \vdash \textbf{undefined} : \texttt{Undefined}}$$

$$\textsc{TypeSeq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 \texttt{ , } e_2 : \tau_2} \qquad \textsc{TypePrint}\ \frac{\Gamma \vdash e_1 : \tau_1}{\Gamma \vdash \texttt{console.log(}e_1\texttt{)} : \texttt{Undefined}} \qquad \textsc{TypeVar}\ \frac{}{\Gamma \vdash x : \Gamma(x)}$$

$$\textsc{TypeConstDecl}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{const}\ x \texttt{ = } e_1 \texttt{; } e_2 : \tau_2}$$

Figure 2: Typing of base TypeScripty, including numbers with arithmetic expressions, booleans with logic and comparison expressions, strings with concatenation, **undefined** with printing, and **const**-variable declarations.

```
type TEnv = Map[String, Typ]
def hastype(tenv: TEnv, e: Expr): Typ = ???
```

Immutable Objects (Records)

Next, we extend our interpreter implementation for immutable objects. We consider the implementation for immutable objects next, as it is a bit simpler than that for multi-parameter functions.

Small-Step Reduction

We extend the reduction-step judgment form $e \longrightarrow e'$ for immutable objects:

$$\boxed{e \longrightarrow e'}$$

$$\textsc{DoGetField}\ \frac{}{\{f_1 : v_1 , \ldots , f_i : v_i , \ldots , f_n : v_n\}.f_i \longrightarrow v_i}$$

$$\textsc{SearchObject}\ \frac{e_i \longrightarrow e_i' \qquad e_j = v_j \qquad \text{for all } j < i}{\{f_1 : e_1 , \ldots , f_i : e_i , \ldots\} \longrightarrow \{f_1 : e_1 , \ldots , f_i : e_i' , \ldots\}}$$

$$\textsc{SearchGetField}\ \frac{e_1 \longrightarrow e_1'}{e_1 . f \longrightarrow e_1' . f}$$

Figure 3: Small-step operational semantics of TypeScripty with immutable objects.

Exercise 0.3 (Small-Step Reduction for Immutable Objects)**.** Implement the cases in `step` for reducing immutable object expressions using the rules given in Figure 3 for the reduction-step judgment form $e \longrightarrow e'$.

Notes

- Field names f are different than variable names x, even though they are both represented in Scala with a `String`. Object expressions are not variable binding constructs—what does that mean about `substitute` for them?
- For SearchObject, you should make the reduction step apply to the first non-value as given by the left-to-right iteration of the collection using the `find` method on `Map`s:

```
(m: Map[K,V]).find(f: ((K,V)) => Boolean): Option[(K,V)]
```

- Other helpful Scala library methods not previously mentioned to use here include the following:

```
(m: Map[K,V]).get(k: K): Option[V]
```

Static Type Checking

We extend the static typing judgment form $\Gamma \vdash e : \tau$ for immutable objects:

$$\textsc{TypeObject}\ \frac{\Gamma \vdash e_i : \tau_i \qquad \text{for all } i}{\Gamma \vdash \{\ldots, f_i : e_i, \ldots\} : \{\ldots, f_i : \tau_i, \ldots\}} \qquad \textsc{TypeGetField}\ \frac{\Gamma \vdash e : \{\ldots, f : \tau, \ldots\}}{\Gamma \vdash e.f : \tau}$$

Figure 4: Typing of TypeScripty with immutable objects.

Exercise 0.4 (Static Type Checking for Immutable Objects)**.** Implement the cases in `hastype` for typing immutable object expressions using the rules given in Figure 4 defining the typing judgment form $\Gamma \vdash e : \tau$.

Notes

- Other helpful Scala library methods not previously mentioned to use here include the following:

```
(m: Map[K,V]).map(f: ((K,V)) => (J,U)): Map[J,U]
```

Multi-Parameter Recursive Functions

Finally, we extend our interpreter implementation for multi-parameter recursive functions.

Small-Step Reduction

We extend the reduction-step judgment form $e \longrightarrow e'$ for multi-paramter recursive functions:

Exercise 0.5 (Small-Step Reduction for Multi-Parameter Recursive Functions)**.** Implement the cases in `step` for reducing multi-parameter recursive functions using the rules given in Figure 5 for the reduction-step judgment form $e \longrightarrow e'$.

$\boxed{e \longrightarrow e'}$

$$\textsc{DoCall}\quad \frac{}{((y_1:\tau_1,\ldots,y_n:\tau_n)\tau^? \Rightarrow e)(v_1,\ldots v_n) \longrightarrow [v_1/y_1]\cdots[v_n/y_n]e}$$

$$\textsc{DoCallRec}\quad \frac{v = (x(y_1:\tau_1,\ldots,y_n:\tau_n):\tau' \Rightarrow e)}{v(v_1,\ldots v_n) \longrightarrow [v/x][v_1/y_1]\cdots[v_n/y_n]e}$$

$$\textsc{SearchCall1}\quad \frac{e \longrightarrow e'}{e(e_1,\ldots,e_n) \longrightarrow e'(e_1,\ldots,e_n)}$$

$$\textsc{SearchCall2}\quad \frac{e_i \longrightarrow e_i' \qquad e_j = v_j \qquad \text{for all } j < i}{v(e_1,\ldots,e_i,\ldots,e_n) \longrightarrow v(e_1,\ldots,e_i',\ldots,e_n)}$$

Figure 5: Small-step operational semantics for TypeScripty with multi-parameter recursive functions.

Notes

- Other helpful Scala library methods not previously mentioned to use here include the following:

```
(l: List[A]).map(f: A => B): List[B]
(l: List[A]).exists(f: A => Boolean): Boolean
(la: List[A]).zip(lb: List[B]): List[(A,B)]
(l: List[A]).forall(f: A => Boolean): Boolean
(l: List[A]).foldRight(f: (A,B) => B): B
```

  - You may want to use the `zip` method for the DoCall and DoCallRec cases to match up formal parameters and actual arguments.

- You might want to use your `mapFirst` function from Homework 4 here.

Static Type Checking

We extend the static typing judgment form $\Gamma \vdash e : \tau$ for multi-parameter recursive functions:

Exercise 0.6 (Static Type Checking for Immutable Objects)**.** Implement the cases in `hastype` for typing multi-parameter recursive function expressions using the rules given in Figure 6 defining the typing judgment form $\Gamma \vdash e : \tau$.

$$\textsc{TypeCall}\quad \frac{\Gamma \vdash e : (y_1 : \tau_1, \ldots, y_n : \tau_n) \Rightarrow \tau \qquad \Gamma \vdash e_1 : \tau_1 \quad \cdots \quad \Gamma \vdash e_n : \tau_n}{\Gamma \vdash e(e_1, \ldots, e_n) : \tau}$$

$$\textsc{TypeFunction}\quad \frac{\Gamma, y_1 : \tau_1, \cdots, , y_n : \tau_n \vdash e' : \tau'}{\Gamma \vdash (\overline{y : \tau}) \Rightarrow e' : (\overline{y : \tau}) \Rightarrow \tau'}$$

$$\textsc{TypeFunctionAnn}\quad \frac{\Gamma, y_1 : \tau_1, \cdots, , y_n : \tau_n \vdash e' : \tau'}{\Gamma \vdash (\overline{y : \tau}) : \tau' \Rightarrow e' : (\overline{y : \tau}) \Rightarrow \tau'}$$

$$\textsc{TypeFunctionRec}\quad \frac{\Gamma, x : \tau_x, y_1 : \tau_1, \cdots, , y_n : \tau_n \vdash e' : \tau' \qquad \tau_x = (\overline{y : \tau}) \Rightarrow \tau'}{\Gamma \vdash x(\overline{y : \tau}) : \tau' \Rightarrow e' : \tau_x}$$

Figure 6: Typing of TypeScripty with multi-parameter recursive functions.

Notes

- Other helpful Scala library methods not previously mentioned to use here include the following:

```
(l: List[A]).foldLeft(f: (B,A) => B): B
(l: List[A]).foreach(f: A => Unit): Unit
(l: List[A]).length: Int
(m1: Map[K,V]).++(m2: Map[K,V]): Map[K,V]
```

  - The ++ method on `Map`s appends two `Map`s together.